Intent OMS — Self-Optimizing Order Orchestration: Requirements

Oct 18, 2026 · @Pradeep

1. Overview

Self-Optimizing Orchestration (SOO) turns Intent OMS from a system that follows configured policy into one that measures every outcome and proposes better policy. It also proves each change is better on replayed and live traffic, then promotes it within guardrails. It covers five areas:

- **Fulfillment rate**: more lines shipped complete and on time.
- **Inventory usage**: less stranded stock and fewer avoidable backorders.
- **Customer satisfaction**: honest promises and fewer unwanted splits.
- **Fraud and abuse**: lower losses without hurting good customers.
- **Cost to serve**: lower cost per order at the same or better service.

Problem. Today every policy in Intent OMS is written by a person and stays fixed until a person changes it. That covers sourcing weights, split rules, safety stock, decision thresholds and reservation TTLs. Networks, carriers, customers and fraud patterns drift weekly, so a fixed policy decays from the day it ships. The data needed to correct it already exists in the event ledger, the decision records and the human task queue. Nothing turns that data back into policy today.

What "learns by itself" means here. The LLM's weights do not change in production. Learning happens in three bounded places, each versioned, testable and reversible:

1. **Parameters**: numbers the system tunes from outcomes. Examples are node reliability, promise calibration, band thresholds, safety stock and policy-selection probabilities.
2. **Knowledge**: typed, versioned facts about segments, customers, nodes and lanes, derived from outcomes. Examples are "this customer cluster tolerates splits" and "this pincode has high refused-delivery risk".
3. **Policy**: rule and decision changes that a typesafe LLM drafts as changesets. These pass the existing lint, golden-set, impact-diff and approval gates before going live.

Statistics decide whether a change is better; the LLM proposes and explains but never declares a win. This is the line that keeps the claim defensible to an enterprise buyer.

2. Goals, non-goals and metrics

Success means each tenant's composite objective (section 7) improves quarter on quarter. No guardrail metric may get worse while that happens.

Goals

- G1. Close the loop from outcome to policy for sourcing, promise, inventory, customer and fraud decisions.
- G2. Make every learned change a typed, versioned artifact that can be replayed, explained and rolled back.
- G3. Let tenants pick an autonomy level per learner, from observe-only to autonomous within caps.
- G4. Reach a measurable improvement within 30 days of connecting a tenant's order history, using replay before any live change.

Non-goals

- Fine-tuning or retraining LLM weights in the production path.
- Cross-tenant learning without explicit tenant opt-in.
- Autonomous changes to refunds, cancellations or pricing. These stay excluded, as they are in the Control Tower today.
- Replacing demand forecasting or WMS slotting. SOO consumes forecasts; it does not build them.

Metrics (targets are starting proposals, to be set per tenant from its replay baseline)

Metric	Definition	Type	Proposed target
Perfect order rate	Lines delivered complete, on time, undamaged, not returned for fulfillment reasons	North star	+3 pts vs baseline in 90 days
Fill rate	Units shipped ÷ units ordered, first attempt	Primary	+2 pts
Promise accuracy	Deliveries on or before promised date	Primary	≥ 95%
Promise calibration error	Gap between stated and actual on-time probability	Primary	≤ 3 pts
Split-shipment rate	Orders shipped in more than one package when one was feasible	Primary	−20%

Metric	Definition	Type	Proposed target
Cost to serve	Fulfillment + shipping + handling per order	Primary	−5%
Inventory turns / stranded units	Units with no sale in N days at a node with demand elsewhere	Primary	−15% stranded
Fraud and abuse loss	Chargebacks + return abuse + refused-delivery losses, as % of GMV	Primary	−25%
False-positive rate	Good orders held or declined by fraud decisions	Guardrail	Must not rise
Override rate	Autonomous actions reversed by a human	Guardrail	≤ 5% (existing KPI)
Customer contact rate	WISMO and complaint contacts per 100 orders	Guardrail	Must not rise
Decision latency	p95 of a decide call	Guardrail	≤ 300 ms (matches promise service)

3. Design principles

Eight principles govern every requirement below; when a requirement conflicts with one, the principle wins.

1. **Typesafe first.** Every LLM input and output is a declared type with enums, ranges and required fields. Free text never drives an action. Untyped output is rejected, repaired once, or falls back to the deterministic provider.
2. **Learn bounded, not unbounded.** Every learner has an explicit action space, an exposure cap and a change budget. A learner can only move what its type allows it to move.
3. **Every change is an artifact.** A learned change is a versioned policy, parameter set or knowledge record with an author, a diff and evidence. Nothing is edited in place.
4. **Prove before promote.** Offline replay, then shadow, then canary, then full rollout. The tier of the decision sets how many steps it needs and who approves.
5. **Statistics decide; LLMs propose.** The promotion gate uses counterfactual estimates and confidence intervals. LLM reasoning is evidence for a human reviewer, never the pass criterion.

6. **Reversible by default.** Every promoted change has a one-click rollback and an automatic rollback trigger on guardrail breach.
7. **Explainable to three audiences.** An ops user sees why the policy changed. A customer sees a customer-safe reason. An auditor sees the full evidence trail.
8. **Tenant-isolated learning.** Each tenant's data trains only that tenant's learners. Shared priors are opt-in and use aggregated, non-identifying statistics only.

4. Core concepts

Ten terms carry the design; the requirements use them exactly as defined here.

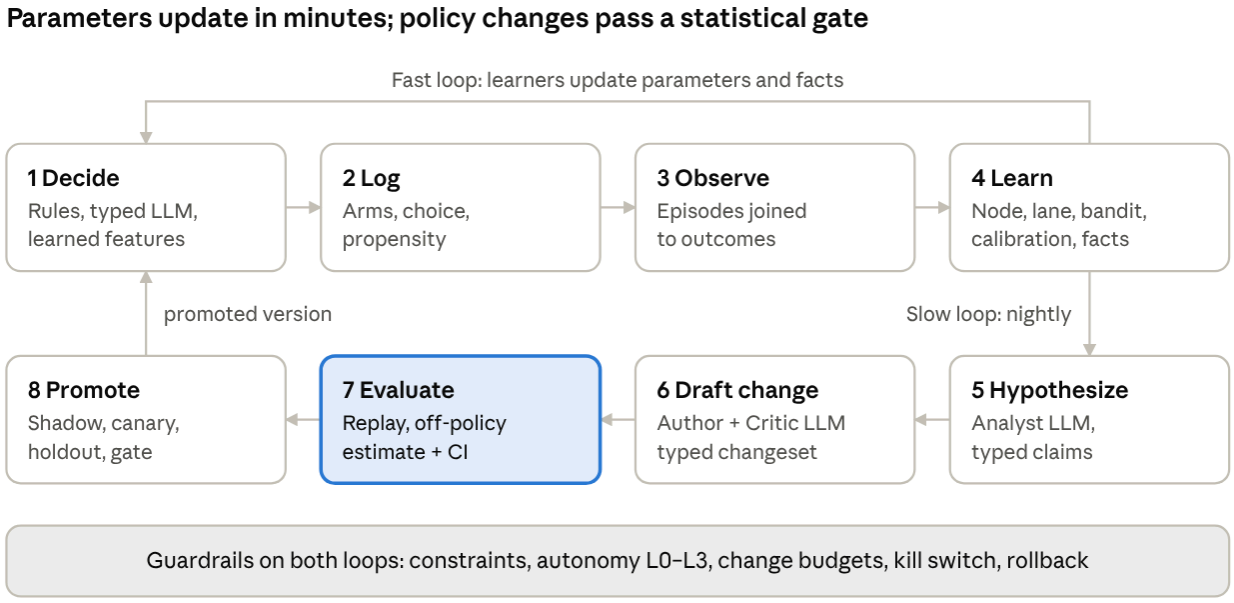
Term	Meaning	Example
Episode	One decision point plus everything that followed it, joined into one record	Sourcing decision for order 1042 → shipped from 2 nodes → delivered 1 day late → customer contacted
Outcome	A typed, measured result attached to an episode once it is known	`DeliveryOutcome{onTime:false, lateByHours:26, damaged:false}`
Attribution	Linking an outcome back to the decisions and parameters that caused it	Late delivery attributed 70% to carrier lane, 30% to node pick delay
Arm	One choice a learner can make at a decision point	Policy `ship-complete`, or node weight set v12
Propensity	The probability the system gave the chosen arm when it chose it	0.80 for `ship-complete`, 0.20 for `split-up-to-2`
Learner	A component that updates parameters or knowledge from outcomes	Node reliability learner, promise calibrator
Proposal	A typed changeset drafted by a learner or the LLM, not yet live	Raise `split_acceptability` act-band threshold from 0.80 to 0.85
Experiment	A controlled exposure of a proposal to a slice of traffic	10% canary for 7 days, holdout of 5%
Promotion	A proposal becoming the live version after passing its gate	v13 replaces v12; v12 kept for rollback

Term	Meaning	Example
Autonomy level	How far a learner may go without a human	L0 observe · L1 recommend · L2 act with approval · L3 autonomous within caps

Propensity logging is the single most important new data point. Without it, past decisions cannot be fairly replayed against a new policy, and every offline estimate is biased.

5. Architecture

SOO wraps the existing order path in two loops. The fast loop tunes numbers from outcomes in minutes with no LLM involved. The slow loop uses the typesafe LLM nightly to propose policy changes, which reach production only through a statistical gate.



learning loop · 8 steps, fast and slow loops

The fast loop moves parameters inside ranges already approved. The slow loop changes the policy itself, so every one of its changes passes replay, shadow and canary first. A rejected or rolled-back change returns to step 5 as evidence against its hypothesis.

5.1 Where each component lives

New component	Lives in	Reads from	Writes to
Decision logger	Decision fabric (extend)	Decision context	`decision_log`
Episode assembler	`oms-learning` (new)	Event ledger, `decision_log`	`episode`, `outcome`

New component	Lives in	Reads from	Writes to
Parameter learners: node, lane, bandit, calibration, safety stock	`oms-learning`	`episode`	Parameter versions read by sourcing, promise and inventory
Fact learners and segment discovery	`oms-learning`	`episode`, `outcome`	`learned_fact`, `segment`
Analyst, author and critic roles	`oms-copilot` (extend)	`OutcomeDigest` aggregates	`hypothesis`, `proposal`
Evaluator	`oms-simulation` (extend)	`decision_log`, `proposal`	Proposal evaluation report
Experiment manager	`oms-learning`	`proposal`	`experiment`, plus a routing table the decision fabric reads
Promotion and rollback	Decision publish path (extend)	Approved proposal	New artifact version, event ledger
Guardrails	`orchestration.autonomy` (extend)	All of the above	Blocks, pauses, kill switch

6. Typesafe LLM layer

The LLM is the system's analyst and author, not its database or its judge. It works only through Java-declared types, which compile to JSON Schema for constrained decoding and are re-validated on the way back.

6.1 Four LLM roles

Role	Input type	Output type	May it act?
Decider (exists: L1/L2)	`DecisionContext<T>`	`Decision<T>` with confidence	Yes, through the existing band and guardrail path

Role	Input type	Output type	May it act?
Analyst (new)	`OutcomeDigest` (aggregates, never raw PII)	`Hypothesis` list	No. Hypotheses go to the statistics engine
Author (extends policy copilot)	`Hypothesis` + current artifact	`PolicyChangeset`	No. Changesets go to the promotion gate
Critic (new)	`PolicyChangeset` + evaluation report	`Review{verdict, risks[]}`	No. Its review is evidence for the approver

6.2 Type contracts

Every contract is a sealed Java type. A sketch of the core set:

```
// The analyst's only output: a testable claim, never a conclusion
record Hypothesis(
    HypothesisId id,
    Scope scope,                    // tenant, segment, node, lane, SKU class
    Metric targetMetric,            // enum: FILL_RATE, PROMISE_ACCURACY,
SPLIT_RATE, ...
    Direction expectedDirection,    // enum: INCREASE, DECREASE
    LeverRef lever,                 // which parameter or rule it would move
    @Range(min = 0, max = 1) double priorConfidence,
    List<EvidenceRef> evidence,     // ids of aggregates the claim rests on
    @MaxLength(280) String rationale
) {}

// The author's only output: a diff against a versioned artifact
sealed interface PolicyChange permits
    ThresholdChange, WeightChange, RuleAdd, RuleRetire, ParameterChange,
AllowListChange {}

record ThresholdChange(
    DecisionName decision, Band band,
    @Range(min = 0, max = 1) double from,
    @Range(min = 0, max = 1) double to
) implements PolicyChange {}

record PolicyChangeset(
    ArtifactRef target, int baseVersion,
    List<PolicyChange> changes,
    HypothesisId motivatedBy,
```

```
    RiskTier tier
) {}

// Learned knowledge, stored as typed facts with provenance
record SegmentFact(
    SegmentId segment, FactKey key,       // enum: SPLIT_TOLERANCE,
SPEED_PREFERENCE, RTO_RISK, ...
    TypedValue value,                     // bounded by FactKey's declared
range
    @Range(min = 0, max = 1) double confidence,
    int sampleSize, Instant learnedAt, LearnerRef source
) {}
```

6.3 Enforcement pipeline

1. **Schema generation.** Java types are compiled to JSON Schema at build time. The schema hash is stored on every LLM call record.
2. **Constrained decoding.** The provider is asked for schema-conformant output (structured outputs, tool-call schema, or grammar). Providers that cannot do this are wrapped with validate-and-repair.
3. **Re-validation.** Output is deserialized into the Java type. Range, enum, cross-field and reference checks run, for example that `LeverRef` exists and `from` matches the live value.
4. **Semantic checks.** Domain rules run next: a threshold change cannot cross another band, and a weight vector must sum to 1.
5. **One repair attempt.** Validation errors are returned to the model as typed errors. A second failure falls back to the deterministic provider or drops the proposal.
6. **Provenance.** Each record stores provider, model, prompt version, schema hash, token cost and latency.

6.4 Rules for LLM inputs

- PII never reaches a model. Inputs use tokenized ids and aggregates from the existing PII vault.
- Customer and agent free text is wrapped as untrusted data and screened by `injection_screen` first. This already happens for the copilot.
- The analyst sees `OutcomeDigest` aggregates with minimum cell sizes (default 30 episodes), so it cannot overfit to one order.
- Every prompt is a versioned artifact under the same lint, golden-set and approval gates as decisions.

7. Objective function and hard constraints

Every learner optimizes one per-episode reward. The reward is built from tenant-weighted outcome terms, and hard constraints are never traded for reward.

```
R(e) = w_{fill} \cdot \text{FillRate}(e) + w_{otd} \cdot \text{OnTime}(e) - w_{cost} \cdot \frac{\text{Cost}(e)}{\text{OrderValue}(e)} - w_{split} \cdot \text{Splits}(e) - w_{fraud} \cdot \text{FraudLoss}(e) - w_{cx} \cdot \text{Contacts}(e) + w_{inv} \cdot \text{AgedStockRelief}(e)
```

- **Weights** are a typed `ObjectiveProfile` per tenant, versioned like any policy. Presets are offered ("margin first", "speed first", "balanced"). A tenant can describe its priorities in plain English and the policy copilot drafts the profile.
- **Normalization.** Every term is scaled to [0, 1] against the tenant's own replay baseline, so weights mean the same thing across tenants.
- **Delayed terms.** On-time and contacts resolve in days; returns and chargebacks resolve in weeks. Episodes carry provisional rewards that are corrected when late outcomes land (FR OC-04).

Hard constraints (a proposal that breaks one is rejected before evaluation):

Constraint	Source
Margin floor per order	Existing margin-floor gate
Compliance holds (hazmat, age, restricted zones)	Existing compliance engine, fail-closed
Refund, cancel and pricing actions never autonomous	Existing Control Tower allow-list rule
Daily action count and dollar-impact caps	Existing guardrails
Node capacity and carrier cut-off times	Node and carrier configuration
No protected attributes as features; pincode-level restrictions need a reason code and review	New fairness rule (GV-07)

8. Functional requirements

There are 63 requirements across ten capability areas. P0 items form the minimum loop that can be sold; P1 deepens it and P2 extends it. "Builds on" names the existing module each one extends.

8.1 Outcome capture and attribution (OC)

ID	Requirement	Priority	Builds on
OC-01	Log every decision with its context features, all candidate arms, the chosen arm, its propensity and the policy version	P0	DecisionService records
OC-02	An episode assembler joins each decision to its downstream events by order, line and fulfillment-group id	P0	Event ledger
OC-03	A typed outcome catalog: `DeliveryOutcome`, `FulfillmentOutcome`, `ReturnOutcome`, `PaymentOutcome`, `ContactOutcome`, `InventoryOutcome`	P0	State machine events
OC-04	Provisional rewards are corrected when late outcomes land. Maturity windows are configurable per type (defaults: delivery 14 days, return 45 days, chargeback 120 days)	P0	—
OC-05	Rule-based attribution splits a bad outcome across causes (stock, node, carrier, customer) with a share per cause	P1	Breach-risk inputs
OC-06	Human resolutions from the task queue are captured as labeled outcomes: what the human did and the reason code	P0	Workflow task queue
OC-07	A data-quality monitor tracks outcome coverage. Learners pause automatically when coverage drops below a threshold (default 80%)	P1	Monitoring

8.2 Customer intelligence (CI)

ID	Requirement	Priority	Builds on
CI-01	Learn typed preference facts per customer and per segment: split tolerance, speed vs cost, pickup preference, contact channel. Facts come from behavior such as complaints after splits, upgrades chosen and returns	P0	`fulfilment_intent`, `split_acceptability`

ID	Requirement	Priority	Builds on
CI-02	Discover segments automatically by clustering behavioral features. The analyst LLM names each segment with a typed label and description; segments are versioned and have a minimum size	P1	—
CI-03	Cold start: new customers inherit segment priors from order context (channel, basket, region), with Bayesian shrinkage toward the prior until evidence builds	P1	—
CI-04	Facts decay with recency using a configurable half-life	P2	—
CI-05	Decisions receive customer facts as typed features in `DecisionContext`, never as raw history	P0	Decision fabric

8.3 Fulfillment strategy learning (FS)

ID	Requirement	Priority	Builds on
FS-01	A contextual bandit picks the sourcing policy per order from the existing five policies, given intent, segment, basket, region and network state. Exploration is capped (default 10% of traffic) and runs only among arms that clear the margin floor and the promise	P0	Sourcing policies
FS-02	A node reliability learner keeps a Beta-binomial estimate per node × SKU class × day-part for on-time pick and reject rate, fed to `sourcing_node_score` as a typed feature	P0	`sourcing_node_score`
FS-03	A lane learner keeps a transit-time distribution per carrier × origin × destination postal code, replacing static transit tables	P0	Promise service
FS-04	The author proposes sourcing-score weight changes as `WeightChange` changesets, validated by replay	P1	Policy copilot

ID	Requirement	Priority	Builds on
FS-05	Learn the backlog-to-breach curve per node and shift load before a node breaches, not after	P1	Breach-risk scoring
FS-06	Replace the fixed store-reject penalty with a learned penalty that decays as the node recovers	P2	Store re-sourcing

8.4 Inventory usage (IU)

ID	Requirement	Priority	Builds on
IU-01	Propose safety stock per SKU × node weekly from demand variance and lead time, within tenant min/max bounds	P1	ATP formula
IU-02	Tune reservation TTLs from observed reservation-to-conversion curves	P1	Reservation sweeper
IU-03	Prefer nodes holding aging stock when the cost difference is small, through the reward's aged-stock term	P1	Sourcing
IU-04	Weekly typed `TransferRecommendation{sku, from, to, qty, splitsAvoided, cost}`. Recommend-only (L1) by default	P2	Margin replay
IU-05	Learn supplier and ASN reliability and adjust how much inbound stock counts toward ATP	P1	Inbound confidence scoring
IU-06	Learn from `backorder_next_action` outcomes (customer kept or cancelled) to tune the action options	P2	`backorder_next_action`

8.5 Promise and customer experience (CX)

ID	Requirement	Priority	Builds on
CX-01	Calibrate the promise's on-time probability per lane, so stated probability matches actual within 3 points	P0	Promise service

ID	Requirement	Priority	Builds on
CX-02	Learn a promise buffer per segment that balances checkout conversion against accuracy (needs a conversion signal from the storefront)	P2	Promise service
CX-03	Learn when to warn customers about a delay: the earliest confident breach prediction that minimizes contacts	P1	Re-promise action
CX-04	Learn the compensation level that minimizes repeat contacts per unit cost, within budget. `appeasement` stays a T2 decision	P2	`appeasement`

8.6 Fraud and abuse (FD)

ID	Requirement	Priority	Builds on
FD-01	Add a pre-capture `payment_risk` decision with learned features: velocity, address and device linkage where available, agent mandate history	P0	Decision fabric, spend guard
FD-02	Add a `rto_risk` decision for refused deliveries in cash-on-delivery markets, with prepaid-conversion and address-confirmation actions	P1	`address_risk`, notifications
FD-03	Tune each fraud decision's act band to a tenant-set false-positive target, weighing expected loss against the cost of blocking a good order	P0	Band thresholds
FD-04	The analyst proposes new abuse rules, such as address-reuse rings, as `RuleAdd` changesets evaluated on labeled history	P1	`return_risk`
FD-05	An agent reputation learner builds a typed `AgentReputation` from mandate compliance, injection-screen hits and dispute rate. It feeds the manual tier change only (L1)	P1	Agent trust tiers
FD-06	Capture fraud labels from chargebacks, return grading and human review outcomes, with label delay handled per OC-04	P0	Returns, finance

ID	Requirement	Priority	Builds on
FD-07	Detect drift in fraud feature distributions; alert and pause auto-tuning when drift exceeds a threshold	P1	Monitoring

8.7 Decision fabric self-tuning (DF)

ID	Requirement	Priority	Builds on
DF-01	Tune band thresholds per decision to maximize accuracy within an escalation budget (human tasks per day)	P0	Four-layer pipeline
DF-02	Calibrate L1 confidence per decision, mapping raw model confidence to observed accuracy	P0	Golden-set calibration error
DF-03	Grow golden sets from human-labeled escalations after deduplication and reviewer sign-off	P0	Golden-set gating
DF-04	Override → Rule: when humans resolve the same situation the same way often enough (default ≥ 20 cases, ≥ 90% agreement), the author drafts a `RuleAdd` or `AllowListChange`	P1	Policy copilot, task queue
DF-05	Learn per decision which provider and model hits the target accuracy at the lowest cost and latency, and route to it	P1	Provider interface
DF-06	The author proposes prompt versions, gated like decision versions	P2	Decision Studio
DF-07	Propose retirement of L0 rules that never fire or that L1 outperforms on replay	P2	Impact-diff tool

8.8 Experimentation and promotion (EX)

ID	Requirement	Priority	Builds on
EX-01	Offline evaluation by replay, using off-policy estimators (inverse propensity and doubly robust) over logged propensities, reported with confidence intervals	P0	`oms-simulation`
EX-02	Shadow mode: the proposal runs in parallel on live traffic and is logged without affecting orders	P0	Decision fabric
EX-03	Canary on a traffic slice, sticky per customer, with a minimum duration and sequential testing for early stop	P0	—
EX-04	A persistent holdout (default 5%) stays on the baseline policy to measure cumulative lift	P0	—
EX-05	Roll back automatically on guardrail breach during or after rollout	P0	One-click rollback
EX-06	Allow only one experiment per decision point per traffic slice	P1	—
EX-07	Promotion gate by tier: T0 promotes automatically after replay and shadow; T1 automatically at autonomy L3, else with approval; T2 always with approval; T3 never autonomous	P0	Tier approvals

8.9 Governance (GV)

ID	Requirement	Priority	Builds on
GV-01	Autonomy level per learner per tenant (L0–L3), default L1	P0	Control Tower guardrails
GV-02	Change budgets: max promotions per decision per week (default 1) and max movement per step (for example, a threshold moves at most 0.05)	P0	—
GV-03	Kill switches at global, per-learner and per-tenant level	P0	Existing kill switches

ID	Requirement	Priority	Builds on
GV-04	Every proposal, evaluation, approval, promotion and rollback is written to the event ledger and covered by its SHA-256 digest	P0	Event ledger, evidence packs
GV-05	Tenant-defined freeze windows (for example, sale days) block promotions	P1	—
GV-06	Crypto-shred deletion propagates to episodes and facts; learners retrain without deleted subjects	P0	PII vault
GV-07	Fairness: protected attributes are excluded; segment-level outcome parity is reported; postal-code-level restrictions need a reason code and human review	P0	—

8.10 Explainability and console (XP)

ID	Requirement	Priority	Builds on
XP-01	A Learning Console screen lists active learners and their autonomy level, the proposals queue, running experiments, promoted changes and lift vs holdout	P0	React console
XP-02	A change card shows the hypothesis, evidence, replay results with confidence intervals, the critic's review and the diff, with approve and reject	P0	Policy copilot review
XP-03	The order detail's "why" drawer also shows which learned facts and policy versions shaped each decision	P0	Decision-trace drawer
XP-04	A weekly learning digest per tenant: what changed, measured impact, what is pending	P1	Notifications
XP-05	Customer-safe explanations through the existing explanation MCP tool	P1	MCP server
XP-06	New ops-copilot query templates over the learning tables	P2	Ops query catalog

ID	Requirement	Priority	Builds on
XP-07	Tenant objective editor: plain-English priorities become a draft `ObjectiveProfile` for review	P1	Policy copilot

9. Data model, events and APIs

SOO adds one module, `oms-learning`, with nine tables, eight events and a REST surface under `/learning`. Every table carries `tenant_id` and follows the existing tenant-context rules.

9.1 Tables

Table	Key columns	Notes
`decision_log`	decision_id, decision_name, version, context_features (jsonb, typed), arms (jsonb), chosen_arm, propensity, policy_version, created_at	Extends today's decision records; append-only
`episode`	episode_id, decision_id, order_id, line_id, status (open/provisional/mature), reward, reward_components (jsonb), matured_at	One row per decision point
`outcome`	outcome_id, episode_id, type, payload (typed jsonb), observed_at, source_event_id	Links back to the event ledger
`learned_fact`	fact_id, subject_type (customer/segment/node/lane/sku), subject_id, fact_key, value, confidence, sample_size, learner, version, valid_from, valid_to	Versioned; never updated in place
`segment`	segment_id, version, label, description, definition (jsonb), size, created_by	Analyst-named, statistics-defined
`hypothesis`	hypothesis_id, scope, target_metric, direction, lever, prior_confidence, evidence_refs, status	Analyst output

Table	Key columns	Notes
`proposal`	proposal_id, changeset (typed jsonb), hypothesis_id, tier, status (draft → evaluated → approved → canary → promoted / rejected / rolled_back), critic_review	Author output
`experiment`	experiment_id, proposal_id, mode (shadow/canary), traffic_share, start, end, stop_rule, result (jsonb: estimate, CI, guardrails)	One active per decision point per slice
`objective_profile`	tenant_id, version, weights (jsonb), constraints (jsonb), approved_by	Section 7

9.2 Events (published to the event ledger)

`EpisodeMatured`, `FactLearned`, `HypothesisRaised`, `ProposalDrafted`, `ProposalEvaluated`, `ExperimentStarted`, `ProposalPromoted`, `ProposalRolledBack`. Each carries the ids above and the actor (learner, LLM role or human).

9.3 REST surface

Method and path	Purpose
`GET /learning/learners`	Learners, autonomy level, status, last run
`PUT /learning/learners/{id}/autonomy`	Set L0–L3 (permission: config-admin)
`GET /learning/facts?subject=…`	Learned facts for a subject, with provenance
`GET /learning/proposals?status=…`	Proposal queue
`POST /learning/proposals/{id}/evaluate`	Run replay evaluation now
`POST /learning/proposals/{id}/approve` · `/reject`	Approval actions (author cannot approve own)
`POST /learning/experiments` · `GET /learning/experiments/{id}`	Start and inspect shadow or canary
`POST /learning/promotions/{id}/rollback`	One-click rollback
`GET /learning/lift`	Cumulative lift vs holdout per metric

Method and path	Purpose
`GET` · `PUT /learning/objective`	Read and update the tenant objective profile
`POST /learning/kill-switch`	Pause all or one learner

9.4 MCP additions

Two read-only tools for internal-ops agents: `learning_status` (what is learning, what changed) and `explain_policy_change` (typed summary of a promotion). No MCP tool can approve or promote.

10. Non-functional requirements

Learning must never slow the order path. All heavy work runs off the request path, and the decide call reads only precomputed facts and parameters.

ID	Area	Requirement
NF-01	Latency	Decide p95 stays ⩽ 300 ms with learned features. Feature lookup adds ⩽ 10 ms p95 from a cache
NF-02	Cadence	Online parameter updates (node, lane, bandit posteriors) within 15 minutes of an outcome. Analyst and author runs nightly. Safety stock weekly
NF-03	LLM cost	A per-tenant monthly budget for analyst, author and critic calls. When exhausted, learners continue on statistics only
NF-04	Determinism	Any decision is replayable from `decision_log` plus the fact and policy versions it read. Bandit sampling uses a seeded random draw stored with the decision
NF-05	Degradation	With every LLM provider down, the order path runs on L0 rules and the heuristic provider, and parameter learners keep running. Only proposal drafting stops
NF-06	Isolation	Learners, caches and feature stores are partitioned by tenant. A test proves one tenant's outcomes never change another tenant's facts

ID	Area	Requirement
NF-07	Privacy	No raw PII in learning tables or LLM inputs; tokenized ids only, per the existing PII vault
NF-08	Scale	Episode assembly keeps up with 10x the tenant's peak hourly order rate, measured in the published load test (roadmap item 8)
NF-09	Observability	Metrics per learner: update lag, coverage, proposal yield, promotion rate, rollback rate
NF-10	Retention	Decision logs and episodes kept 400 days by default, so a year-on-year peak can be replayed

11. Safety and failure modes

A self-optimizing system fails in predictable ways. Each known way has a detection signal and a built-in response.

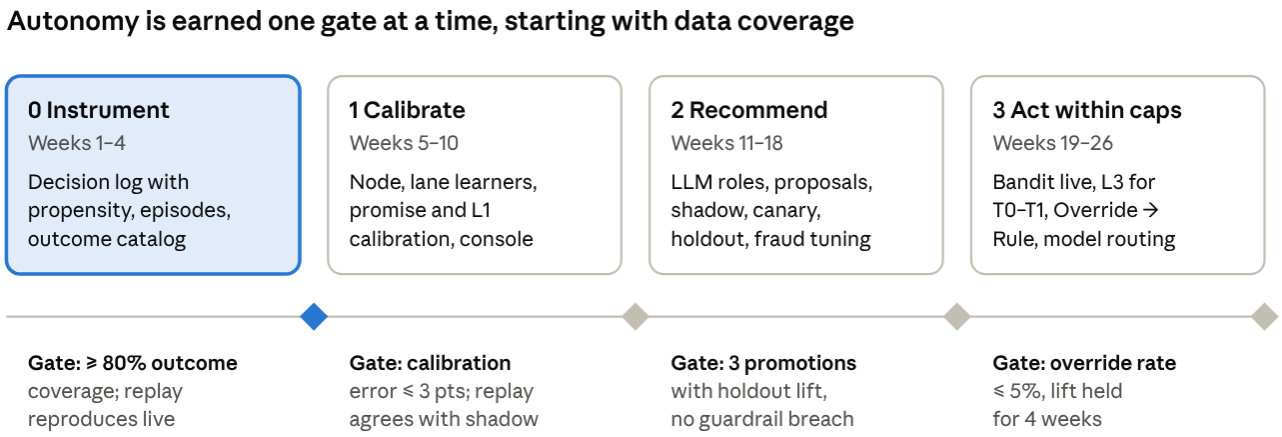
Failure mode	What it looks like	Detection	Response
Reward hacking	The optimizer cuts cost by quietly shipping late or splitting more	Guardrail metrics move while reward rises	Guardrails are hard gates (EX-05); reward terms are reviewed in `objective_profile` approval
Feedback loop	A node marked unreliable gets no traffic, so it can never prove it recovered	Fact confidence stops updating; sample size flat	Minimum exploration share per arm (FS-01) and fact decay (CI-04)
Delayed-label bias	Fraud tuning looks great because chargebacks have not arrived yet	Share of immature episodes in the evaluation window	Promotion waits for maturity (OC-04); reports show the mature share
Distribution shift	Sale day, new carrier, new region break learned facts	Drift monitor on features and outcomes (FD-07, OC-07)	Pause learners, fall back to last stable version, freeze windows (GV-05)

Failure mode	What it looks like	Detection	Response
LLM hallucinated lever	The author proposes a change to a parameter that does not exist or exceeds its range	Type and reference validation (6.3)	Rejected before evaluation; counted in the author's quality metric
Prompt injection via data	Customer text tries to steer the analyst	`injection_screen` on any free text	Analyst sees aggregates only (6.4)
Unfair outcomes	A postal-code or segment is systematically held or slowed	Parity report per segment (GV-07)	Human review; restriction needs a reason code
Runaway changes	Many small promotions compound into a large drift	Change budget counter (GV-02)	Weekly cap; cumulative diff vs last approved baseline shown on every change card
Silent data loss	Webhooks stop arriving, outcomes look "all good"	Coverage monitor (OC-07)	Learners pause below the coverage threshold

Never autonomous, regardless of tenant settings: refunds, cancellations, pricing, objective-profile changes, autonomy-level changes, and any change to a T3 decision.

12. Phased rollout and acceptance criteria

SOO ships in four gated phases over roughly 26 weeks (proposed), on one pilot tenant first. Phase 2 is the first release that can be sold as self-optimizing.



rollout · 4 phases, proposed weeks, gate criteria

No phase starts until the previous gate passes on the pilot tenant. Phase 4 (extend) adds the P2 items and opt-in shared priors once three tenants are live.

Phase	Requirements delivered
0 Instrument	OC-01, OC-02, OC-03, OC-04, OC-06, NF-04, GV-04, GV-06
1 Calibrate	FS-02, FS-03, CX-01, DF-02, EX-01, OC-07, XP-01 (read-only), XP-03
2 Recommend	CI-01, CI-05, FS-01 (shadow), FD-01, FD-03, FD-06, DF-01, DF-03, EX-02, EX-03, EX-04, EX-05, EX-07, GV-01, GV-02, GV-03, GV-07, XP-02, XP-07
3 Act within caps	FS-01 (live), FS-04, FS-05, EX-06, DF-04, DF-05, GV-05, IU-01, IU-02, IU-03, IU-05, CX-03, FD-02, FD-04, FD-05, FD-07, CI-02, CI-03, OC-05, XP-04, XP-05
4 Extend	FS-06, IU-04, IU-06, CX-02, CX-04, CI-04, DF-06, DF-07, XP-06, shared priors

Release-level acceptance (all must hold on the pilot tenant before general availability):

- [ ] Every decision is replayable from logged context, fact versions and policy version (NF-04)
- [ ] Holdout lift on the north-star metric is positive with a 95% confidence interval above zero
- [ ] No guardrail metric worse than baseline over the last 4 weeks
- [ ] Every promoted change has a change card, an evidence trail and a tested rollback

- [ ] With all LLM providers disabled, orders keep flowing and parameter learners keep updating (NF-05)
- [ ] A cross-tenant isolation test passes (NF-06)

13. Risks, open questions and positioning

The biggest risk is not the learning; it is having enough clean outcome data per tenant to learn from. Phase 0 exists to prove that before anything else is built.

Risks

Risk	Mitigation
Small tenants have too few episodes for per-segment learning	Hierarchical priors (CI-03), opt-in shared priors, longer evaluation windows; promise only parameter learning below a volume floor
Outcome data depends on carrier and WMS webhooks the tenant may not send	Coverage monitor (OC-07); onboarding checklist names required feeds; tracking-API polling as fallback
Buyers distrust "self-learning" in a system that moves money	Default autonomy L1, visible holdout lift, evidence packs and one-click rollback in every demo
LLM cost grows with tenant count	Analyst runs on aggregates, nightly; DF-05 routes to cheapest adequate model; NF-03 budget cap
Off-policy estimates are noisy when propensities are near 0 or 1	Exploration floor per arm; clipped estimators; shadow and canary confirm before promotion

Open questions

- Which storefront signal provides checkout conversion for CX-02 (Shopify checkout events, or a tenant-sent event)?
- Should shared priors across tenants be offered at launch, or only after the first 5 tenants are live?
- Which reward weights ship as presets, and who at the tenant approves the objective profile?
- Is the bandit (FS-01) per tenant only, or per tenant × region for large networks?

Positioning: what to claim, and what not to

Say	Avoid
"Intent OMS learns from every order's outcome and proposes better policy, proven on your own history before it goes live."	"The AI trains itself."
"Every learned change is typed, versioned, explained and reversible in one click."	"Fully autonomous" without naming the guardrails
"You choose how much it may change on its own, per decision."	"Black-box optimization"
"Lift is measured against a live holdout, not a slide."	Uplift numbers before a tenant's own replay